



**Instituto Tecnológico de Las Américas
(ITLA)**

Integrantes:

Iván Steven Hernández Peguero (2025-1875)
Abel Quezada Lorenzo (2025-1308)
Allan Sarante (2023-1321)
Alexander Javier De Los Santos Lugo (2025-0924)

Asignatura

Base de Datos Avanzada

Profesor

Daniel Arturo Sánchez De Oleo

Tema

Proyecto final base de datos avanzada (documentación)

Fecha

03 de agosto del año 2026

Santo Domingo Norte, Sto. Dgo., República Dominicana.

Índice

3. Introducción	3
4. Descripción de la empresa y del departamento.	4
5. Requerimientos del negocio.	5
6. Modelo de datos	11
7. Diagrama Entidad-Relación	13
8. Diseño físico de la base de datos (MySQL)	14
9. Diagrama de arquitectura del sistema.....	14
10. Justificación técnica del stock tecnológico	15
11. Vistas, procedimientos almacenados, funciones y triggers.	16
12. Consultas avanzadas SQL.	21
13. Implementación MongoDB.....	25
14. Implementación Redis.....	29
15. Conclusiones.....	32

3. Introducción

El presente documento describe el diseño de un sistema de base de datos para el departamento de Recursos Humanos de Industrial Textil Nacional (ITN). El sistema gestiona el ciclo completo de los colaboradores de la empresa: datos personales y contractuales, nómina, evaluaciones de desempeño, formación y control de asistencia, incorporando además auditoría de cambios sobre datos sensibles e integración con sistemas externos de contabilidad y asistencia.

Dada la naturaleza heterogénea de la información que maneja RRHH, datos altamente estructurados junto a información flexible y variable, además de necesidades de alto rendimiento en tiempo real, el sistema se diseñó bajo una arquitectura de persistencia políglota, combinando tres motores de base de datos: MySQL (relacional), MongoDB (no relacional) y Redis (en memoria), cada uno aplicado a la porción de datos para la que resulta más adecuado.

4. Descripción de la empresa y del departamento.

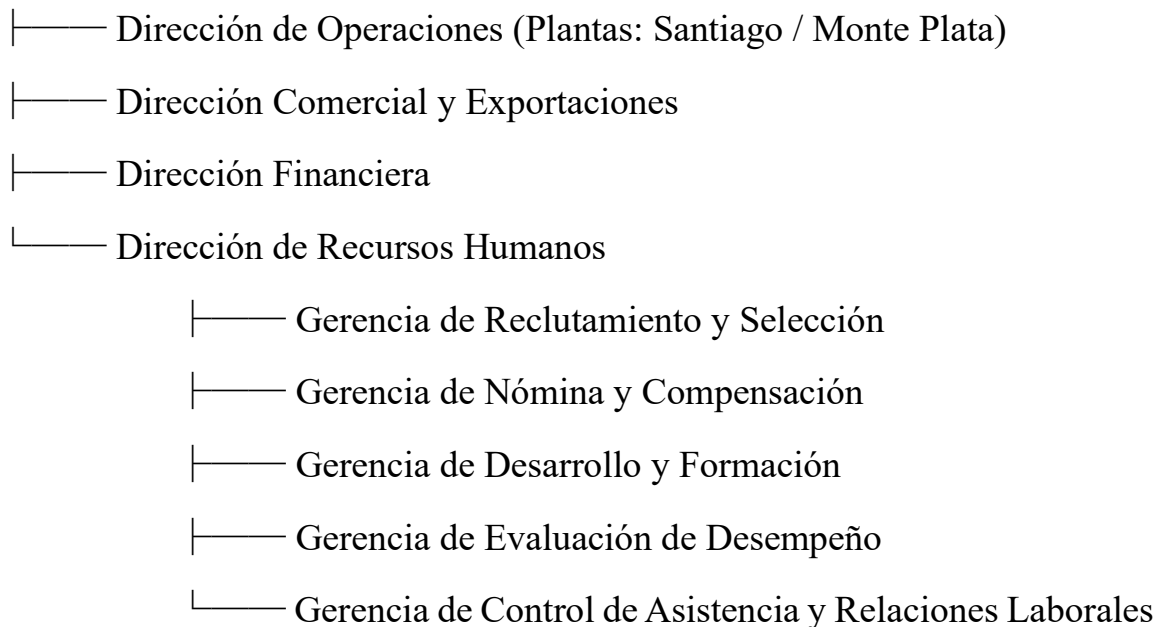
Industrial Textil Nacional (ITN) es una empresa manufacturera dominicana dedicada a la producción y comercialización de textiles (hilados, tejidos y confección) para los mercados nacional y de exportación. Cuenta con más de 10,000 colaboradores distribuidos en tres sedes operativas:

- **Santo Domingo (D.N.):** Sede administrativa y centro de distribución.
- **Santiago:** Planta de producción principal.
- **Monte Plata:** Planta de manufactura y confección.

Misión: Producir textiles de alta calidad que impulsen el desarrollo industrial del país, generando empleo digno, fomentando la innovación en manufactura y contribuyendo al crecimiento sostenible de las comunidades donde opera.

Estructura organizacional relevante al sistema:

Dirección General



El departamento de RRHH está conformado por 100 colaboradores y gestiona cinco procesos principales: reclutamiento y selección, nómina, evaluación de desempeño, formación y capacitación, y control de asistencia. Cada uno a cargo de una gerencia específica.

5. Requerimientos del negocio.

5.1 Descripción funcional del negocio

ITN necesita modernizar y centralizar la gestión de su capital humano mediante una plataforma que administre el ciclo de vida completo de sus colaboradores, desde operarios de planta hasta personal administrativo. Las operaciones diarias exigen control sobre datos personales, catálogos de puestos, tiempo y asistencia (muy necesario para los turnos de producción) y el cálculo exacto de nóminas (salarios, deducciones, impuestos). Adicionalmente, ITN mantiene una fuerte cultura de formación en manejo de maquinaria y seguridad industrial, por lo que requiere almacenar de forma flexible las evaluaciones de desempeño y el historial de capacitación de cada colaborador.

5.2 Actores y roles del sistema

Actor	Rol en ITN	Qué necesita del sistema
Admin RRHH	Opera el día a día: altas, bajas, actualización de datos de empleados en las 3 sedes.	Acceso completo a gestión de empleados/contratos.

Actor	Rol en ITN	Qué necesita del sistema
Gerente de área	Supervisa su gerencia específica.	Reportes de su proceso.
Empleado	Personal operativo o administrativo.	Consultar su propio historial: nómina, evaluaciones, cursos, asistencia.
Auditor	Revisa cumplimiento y trazabilidad de cambios.	Acceso de solo lectura a logs de auditoría.
Contabilidad	Departamento externo a RRHH.	Acceso limitado/exportación de datos de nómina procesada.
Sistema biométrico	Actor técnico.	Alimenta automáticamente el registro de asistencia.

5.3 Procesos cubiertos

Proceso	Qué cubre en ITN
Reclutamiento y selección	Publicación de vacantes por sede, filtro, entrevistas y contratación.
Gestión de empleados y contratos	Alta/baja, datos personales, historial laboral, tipo de contrato.

Proceso	Qué cubre en ITN
Nómina	Pago quincenal/mensual, deducciones legales (TSS, AFP, ISR), horas extra, bonificaciones.
Evaluación de desempeño	Ciclo semestral por objetivos (administrativos) y metas de producción (operativos).
Formación y capacitación	Inducción, capacitación técnica, seguridad industrial, liderazgo.
Control de asistencia	Marcaje biométrico en las 3 sedes, puntualidad, ausentismo, turnos rotativos.
Auditoría de cambios	Trazabilidad de modificaciones sobre datos sensibles.

5.4 Requerimientos funcionales (RF)

- **Gestión de empleados:** registrar, editar, consultar e inactivar (baja) empleados; mantener historial de cambios de cargo, salario y departamento.
- **Gestión de nómina:** calcular y generar la nómina por empleado (salario, deducciones, bonificaciones, horas trabajadas); generar recibos de pago individuales; las deducciones legales (TSS, AFP, ISR) son con tasas fijas a nivel de esquema.
- **Evaluaciones de desempeño:** registrar evaluaciones por periodo con criterios y calificaciones; consultar historial de evaluaciones por empleado.
- **Formación y capacitación:** registrar cursos, capacitaciones y certificaciones por empleado.

- **Auditoría:** registrar automáticamente quién, cuándo y qué se modificó en cualquier dato sensible.
- **Integraciones:** exportar datos de nómina al sistema de contabilidad; importar horas trabajadas y ausencias desde el sistema de tiempo y asistencia.
- **Reportes:** generar y exportar (PDF/Excel) reportes de nómina, rotación de personal, ausentismo, evaluaciones y formación.

5.5 Requerimientos no funcionales (RNF)

Categoría	Requerimiento
Seguridad	Datos sensibles encriptados en reposo y en tránsito.
Seguridad	Acceso restringido por roles
Auditoría	Todo registro de auditoría es inmutable (no editable ni eliminable).
Disponibilidad	Copias de seguridad periódicas y plan de recuperación ante fallos.
Rendimiento	Consultas y reportes responden en menos de 5 segundos, incluso con alto volumen.
Integridad	Consistencia entre módulos integrados (nómina-contabilidad, asistencia-nómina).
Tolerancia a fallos	Manejo de fallas de conexión con sistemas externos sin pérdida de datos.

Categoría	Requerimiento
Cumplimiento normativo	Cumplimiento de la normativa dominicana de protección de datos personales.
Usabilidad	Un usuario nuevo debe poder registrar un empleado o consultar una nómina sin capacitación extensa.

5.6 Reglas de negocio clave

1. Un empleado no puede tener dos contratos activos simultáneamente (incluso entre sedes).
2. Un contrato debe estar asociado a un puesto y departamento vigentes al momento de su creación.
3. Al finalizar un contrato, el empleado pasa a estado inactivo y no aparece en la nómina del periodo siguiente.
4. Un empleado no puede eliminarse físicamente (borrado lógico) para preservar histórico.
5. Cambios de puesto o salario quedan registrados mediante un nuevo contrato (histórico por empleado). No se modela historial aparte para el departamento, ya que se deriva del puesto vigente.
6. Un empleado solo recibe un pago de nómina por periodo.
7. Las deducciones legales (TSS, AFP, ISR) se calculan automáticamente, no se ingresan manualmente.
8. Un pago de nómina no puede procesarse sin contrato activo en ese periodo.
9. Todo ajuste manual a un pago ya generado debe quedar auditado.
10. Un empleado solo puede tener una evaluación por ciclo semestral.

11. Solo el gerente/supervisor directo puede evaluar a un empleado.
12. Una evaluación cerrada no puede modificarse; una corrección genera un nuevo registro.
13. Un empleado no puede inscribirse dos veces a la misma capacitación en el mismo periodo.
14. Las capacitaciones obligatorias quedan vinculadas al puesto.
15. Un marcaje debe estar vinculado a un empleado con contrato activo y a una sede válida.
16. No pueden existir dos marcajes de entrada consecutivos sin una salida entre ambos.
17. Las ausencias médicas tienen nivel de confidencialidad especial.
18. Todo cambio (INSERT/UPDATE/DELETE) sobre datos sensibles queda registrado vía trigger en la tabla de auditoría (usuario, fecha/hora, valor anterior, valor nuevo).
19. Solo el rol Auditor puede consultar la tabla de auditoría.

5.7 Datos clasificados como sensibles

- **Identidad:** cédula/pasaporte, fecha de nacimiento, estado civil.
- **Contacto personal:** dirección particular, teléfono personal, contactos de emergencia.
- **Financieros:** salario base, cuenta bancaria, deducciones de impuestos, historial de auditoría salarial.
- **Médicos:** justificaciones médicas, licencias por enfermedad o maternidad/paternidad.
- **Profesionales privados:** resultados de evaluaciones con comentarios de supervisores, acciones disciplinarias.

5.8 Reportes complejos requeridos

#	Reporte	Origen de datos
1	Nómina consolidada por departamento	MySQL (JOIN + agregación)
2	Histórico de evaluaciones por empleado	MySQL + MongoDB
3	Ausentismo por periodo	MySQL (agregación)
4	Rotación de personal (turnover)	MySQL
5	Cumplimiento de capacitaciones obligatorias	MySQL + MongoDB
6	Ranking de desempeño	Redis (sorted sets)
7	Auditoría de cambios sensibles	MySQL
8	Costo de nómina proyectado vs. real	MySQL

6. Modelo de datos

6.1 Entidades principales (MySQL)

Empleado, Contrato, Departamento, Puesto, Sede, Nómina, Asistencia, Usuario y Auditoría.

El detalle completo de columnas y tipos de cada tabla se encuentra en la sección 8 (Diseño físico).

6.2 Relaciones y cardinalidad

Sede → Departamento (1:N): una sede alberga múltiples departamentos.

Departamento → Puesto (1:N): un departamento tiene múltiples puestos.

Departamento → Empleado (1:1, rol de gerente): un departamento tiene un gerente.

Puesto → Empleado (1:N): un puesto puede ser ocupado por varios empleados.

Empleado → Contrato (1:N): historial de contratos por empleado.

Puesto → Contrato (1:N): el contrato especifica el puesto vigente al momento de contratación. Pueden ser varios puestos.

Empleado → Asistencia (1:N): múltiples marcajes por empleado.

Empleado → Nómina (1:N): múltiples pagos por empleado.

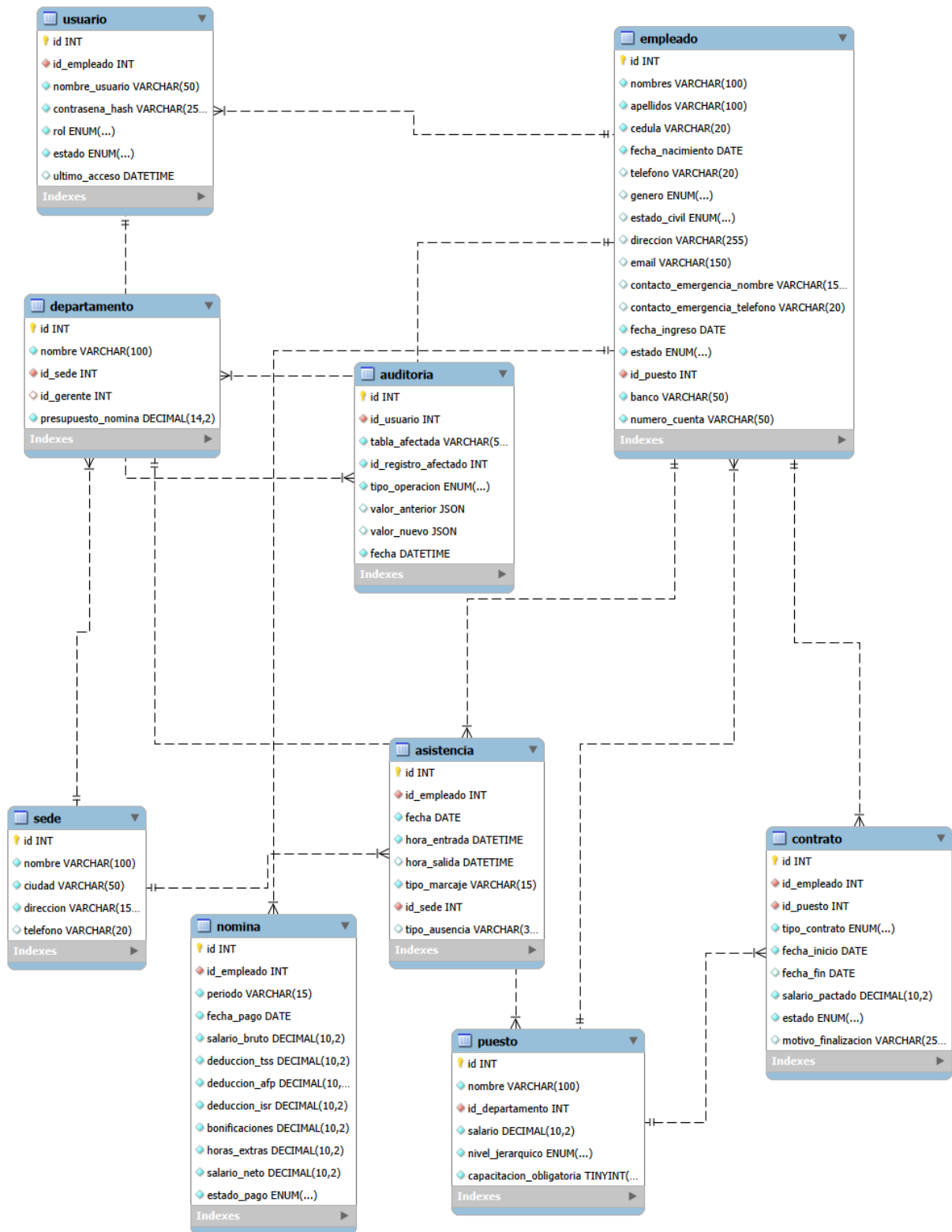
Empleado → Usuario (1:1): un empleado tiene como máximo una cuenta de acceso.

Usuario → Auditoría (1:N): un usuario genera múltiples registros de auditoría.

6.3 Normalización aplicada (hasta 3FN)

Existe una dependencia circular que no fue eliminada de forma intencional (entre departamento, empleado y puesto (gerente → empleado → puesto → departamento)), fue resuelta en estructuraTablas.sql desactivando temporalmente FOREIGN_KEY_CHECKS.

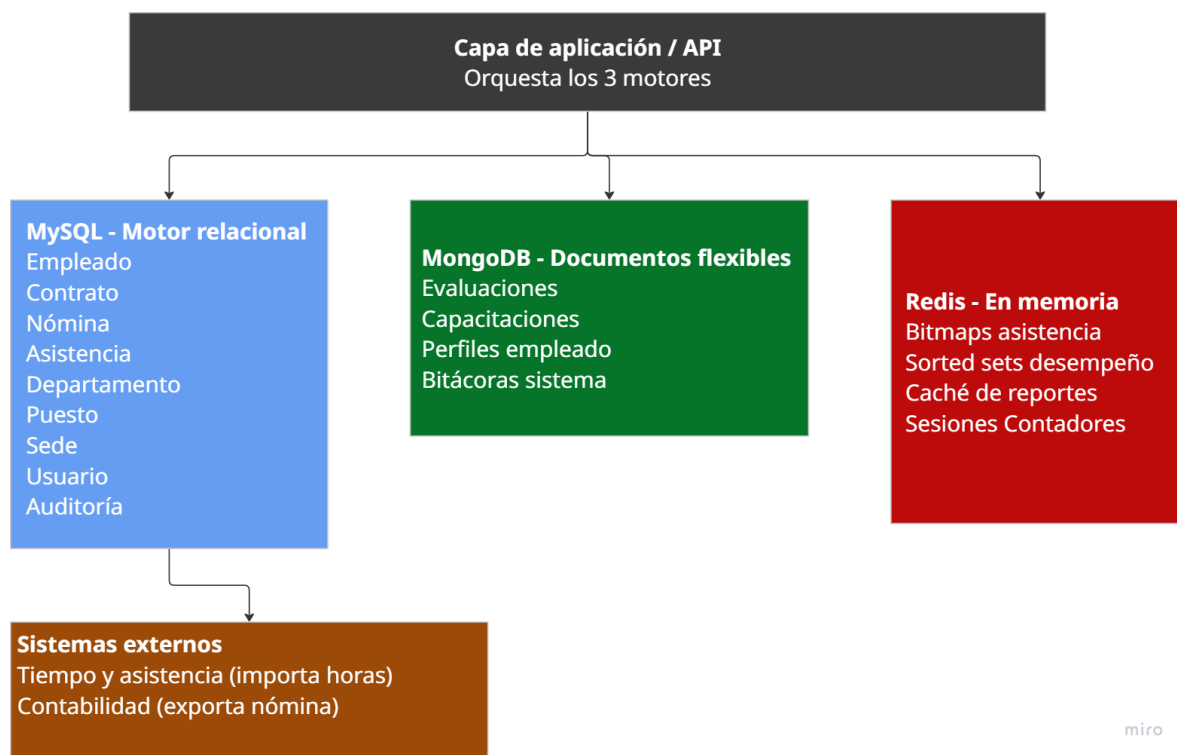
7. Diagrama Entidad-Relación



8. Diseño físico de la base de datos (MySQL)

Tabla	Estado
empleado	Completa
contrato	Completa
asistencia	Completa
nomina	Completa
auditoria	Completa
departamento	Completa
puesto	Completa
sede	Completa
usuario	Completa

9. Diagrama de arquitectura del sistema



10. Justificación técnica del stock tecnológico

Cada motor se eligió según qué tan estructurados son los datos, qué tan crítica es la integridad referencial/transaccional, y qué tan rápido debe responder el sistema.

10.1 MySQL — datos transaccionales con relaciones fuertes

Van aquí: Sede, Departamento, Puesto, Empleado, Contrato, Nómina, Asistencia, Usuario y Auditoría.

Son entidades con estructura fija y relaciones estrictas; el cálculo de nómina exige que salarios, deducciones y totales sean consistentes, lo cual requiere garantías ACID. Las reglas CHECK, UNIQUE y ON DELETE RESTRICT solo existen de forma nativa en un motor relacional. La tabla de Auditoría va en MySQL (y no en Mongo) porque necesita amarrarse por llave foránea exacta a Usuario y quedar inmutable, campo por campo.

10.2 MongoDB — datos con estructura variable o anidada

Colecciones alojadas aquí: evaluaciones, capacitaciones, perfiles_empleado, bitacoras_sistema.

Estas entidades tienen forma irregular o crecen con el tiempo: una evaluación tiene un arreglo variable de criterios según el puesto; una capacitación tiene un arreglo de inscripciones con distintos estados por empleado; el perfil extendido no se llena igual para todos los empleados.

bitacoras_sistema registra acciones generales del sistema (login, exportación de reportes), distinto de la tabla de Auditoría, que registra el cambio campo por campo de un registro específico.

10.3 Redis — datos efímeros o de alta velocidad

Casos de uso definidos: **Bitmaps** para el control de asistencia biométrica (permite calcular ausentismo/puntualidad de miles de empleados con BITCOUNT en milisegundos) y **Sorted Sets** para el ranking de desempeño por sede (obtiene el top-N de empleados sin recalcular sobre MySQL).

Adicionalmente se contempla el uso de caché con expiración para reportes pesados y contadores derivados (empleados activos por sede, capacitaciones pendientes por puesto).

11. Vistas, procedimientos almacenados, funciones y triggers.

11.1 Enfoque general

Mientras que las tablas garantizan la estructura y la integridad de los datos, estos cuatro tipos de objetos son la **capa de lógica de negocio dentro del propio motor**: convierten reglas que de otra forma vivirían solo en el código de la aplicación en restricciones y comportamientos que MySQL aplica siempre, sin importar desde dónde se escriba en la base de datos. Esto es especialmente relevante en RRHH porque varias reglas de negocio (un solo contrato activo, un solo pago por periodo, auditoría de cambios sensibles) son reglas de **integridad**, no solo de interfaz.

11.2 Vistas

Una vista no almacena datos: es una consulta guardada que se comporta como una tabla de solo lectura. Se usaron para dos propósitos: exponer solo lo necesario según el rol que consulta, y dejar pre-armado el nivel de detalle que un reporte necesita sin repetir el mismo JOIN cada vez que alguien lo pide.

vista_empleado_detalle

¿Qué resuelve? Es la vista base de consulta operativa: junta empleado, puesto, departamento y sede en un solo registro (nombre, cédula, estado, puesto, nivel jerárquico, departamento, sede, ciudad). Sirve como punto de partida para cualquier pantalla o reporte que necesite el contexto completo de un empleado sin reescribir ese JOIN de cuatro tablas cada vez.

vista_contratos_activos

¿Qué resuelve? Filtra directamente los contratos con estado = 'ACTIVO', que es la condición que se consulta constantemente para saber quién está trabajando ahora, en qué puesto y bajo qué salario pactado, sin repetir ese WHERE en cada consulta de nómina o de gestión de personal.

vista_nomina_departamento

¿Qué resuelve? El Reporte #1 de la sección 5.8. Agrega los pagos de nómina por departamento y periodo (cantidad de empleados, total bruto, total de deducciones, total de bonificaciones/extras y total neto), excluyendo los pagos anulados. Es el nivel de detalle que un gerente de área necesita para revisar el costo de su equipo, sin ver el detalle individual de cada empleado.

vista_directorio_empleados

¿Qué resuelve? RNF02 exige que el acceso esté restringido por rol y que no todos deban ver salario, cédula o datos bancarios. Esta vista expone únicamente nombre completo, contacto, puesto, departamento y sede de los empleados activos, ocultando por diseño cualquier dato sensible, para que un rol como Empleado o Gerente pueda consultar el directorio sin exponer información financiera o personal.

vista_reporte_nomina_contabilidad

¿Qué resuelve? RF11, la integración con contabilidad. En vez de que el sistema externo tenga que hacer sus propios JOINS sobre las tablas internas de RRHH, esta vista entrega ya armado el detalle de cada pago (folio, periodo, fecha de pago, empleado, departamento, bruto, deducciones totales, ingresos extra, neto y estado de pago), listo para exportar.

11.3 Funciones

Una función encapsula un cálculo que se repite en muchos lugares (consultas, reportes, procedimientos), para no reescribir la misma fórmula una y otra vez ni arriesgarse a que dos partes del sistema la calculen distinto.

fn_calcular_antiguedad

¿Qué resuelve? Varias partes del sistema necesitan saber cuántos años lleva un empleado en la empresa (clasificación de personal, reportes, posibles bonos por

antigüedad). Recibe la fecha de ingreso y devuelve la diferencia en años contra la fecha actual, en vez de repetir `TIMESTAMPDIFF(YEAR, fecha_ingreso, CURDATE())` en cada consulta.

fn_salario_vigente

¿Qué resuelve? Como el salario cambia de contrato en contrato (regla de negocio 5), preguntar "¿cuánto gana este empleado ahora?" implica siempre filtrar su contrato con estado = 'ACTIVO'. Esta función centraliza esa búsqueda: recibe el id del empleado y devuelve el salario_pactado de su contrato vigente, para no repetir ese mismo `SELECT ... WHERE estado = 'ACTIVO' LIMIT 1` en reportes, procedimientos o consultas sueltas. El cálculo del neto (con deducciones e ISR por tramos) no vive aquí, sino dentro del procedimiento `sp_procesar_nomina`, descrito abajo.

11.4 Procedimientos almacenados

Un procedimiento agrupa varias operaciones relacionadas en una sola unidad transaccional: o se ejecutan todas, o no se ejecuta ninguna (`START TRANSACTION / COMMIT / ROLLBACK`). Se usan para las operaciones más sensibles del sistema, donde un fallo a mitad de camino dejaría los datos inconsistentes.

sp_registrar_nuevo_contrato

¿Qué resuelve? La regla de negocio 5: un cambio de puesto o salario no se sobrescribe, se registra como un nuevo contrato, manteniendo el anterior como historial. Este procedimiento hace tres cosas como una sola operación atómica:

inactiva el contrato anterior, inserta el nuevo, y sincroniza el puesto vigente en la ficha del empleado. Si se hiciera como tres sentencias sueltas desde la aplicación, un error a mitad de camino podría dejar al empleado con dos contratos activos o con el puesto desincronizado.

sp_procesar_nomina

¿Qué resuelve? Concentra en una sola operación todo el cálculo de un pago de nómina: toma el salario_pactado del contrato activo del empleado, calcula las deducciones legales con tasas fijas (TSS 3.04 %, AFP 2.87 %) y el ISR por tramos según la escala vigente, suma bonificaciones y horas extra, y calcula el neto resultante. Si el empleado no tiene contrato activo, el procedimiento lanza un error controlado y no inserta nada — así se cumple dentro del propio motor la regla de negocio 8 ("un pago de nómina no puede procesarse sin contrato activo"), sin depender de que la aplicación lo valide antes de llamarlo.

sp_dar_baja_empleado

¿Qué resuelve? La regla de negocio 4: un empleado no se elimina físicamente, se da de baja lógica. Este procedimiento inactiva el contrato vigente del empleado (con fecha de fin y motivo) y cambia su estado a INACTIVO, como una sola operación transaccional. Es el mecanismo real detrás de "dar de baja" a alguien sin perder su historial de contratos, nómina y asistencia.

11.5 Triggers

Un trigger se ejecuta automáticamente ante un evento (INSERT/UPDATE) sobre una tabla, sin que nadie tenga que invocarlo explícitamente. Se usan para lo que se debe cumplir siempre, sin excepción, y para la auditoría, que por

definición no se le puede confiar solo a la aplicación (RNF03: los registros de auditoría deben ser inmutables e independientes de quién los dispare).

trg_valida_contrato_activo_unico

¿Qué resuelve? La regla de negocio 1: un empleado no puede tener dos contratos activos simultáneamente. Se dispara antes de insertar un contrato y, si detecta que ya existe uno activo para ese empleado, bloquea la inserción con un error — es la diferencia entre una regla que "se supone" que la aplicación respeta, y una que la base de datos impone sin excepción, incluso si alguien intenta insertar directamente por SQL.

trg_auditoria_contrato

¿Qué resuelve? La regla 18 (todo cambio sobre datos sensibles queda auditado). Cada vez que cambia el estado o el salario_pactado de un contrato, se inserta automáticamente un registro en auditoria con el valor anterior y el nuevo en formato JSON, sin que quien hizo el cambio pueda evitarlo.

trg_auditoria_empleado

¿Qué resuelve? La misma regla 18, aplicada al empleado: cada vez que cambia su estado (por ejemplo, al darlo de baja) o su puesto vigente, queda registrado en auditoria con el valor anterior y el nuevo. Entre los dos triggers de auditoría queda cubierto tanto el historial contractual como la ficha del empleado, que son los dos puntos donde vive la información sensible según 5.7.

12. Consultas avanzadas SQL.

12.1 Enfoque general

Estas consultas no crean ni modifican objetos: son ejemplos de explotación de los datos usando técnicas avanzadas de SQL — CTEs simples y recursivas, funciones de ventana, subconsultas correlacionadas, EXISTS, self-joins,

agregación condicional y WITH ROLLUP — para responder preguntas de negocio reales de RRHH que van más allá de lo que cubren las vistas fijas. El primer bloque responde directamente a los reportes complejos de la sección 5.8; el segundo resuelve análisis operativos puntuales que no ameritan una vista permanente.

12.2 Nómina por departamento con variación mensual

¿Qué resuelve? Agrega la nómina neta por departamento y periodo, y usa LAG() para comparar cada periodo contra el inmediato anterior del mismo departamento, mostrando el porcentaje de variación. Es el complemento de vista_nomina_departamento: no solo la foto de un periodo, sino la tendencia mes a mes.

12.3 Costo de nómina proyectado vs. real por departamento

¿Qué resuelve? Compara, periodo por periodo, el presupuesto_nomina de cada departamento contra el gasto real (SUM(salario_bruto)), y calcula el porcentaje ejecutado con una etiqueta de estado (dentro de presupuesto / cerca del límite / excede presupuesto).

12.4 Departamentos que superaron su presupuesto de nómina

¿Qué resuelve? Complementa la consulta anterior con una subconsulta correlacionada (EXISTS): en vez de mostrar el detalle periodo a periodo, lista directamente qué departamentos llegaron a excederse en algún momento, útil para una alerta rápida.

12.5 Ausentismo por empleado y departamento en un periodo.

¿Qué resuelve? Es la consulta más elaborada del script: una CTE recursiva genera el calendario de días hábiles del periodo (excluyendo fines de semana), se cruza contra los empleados activos y se compara contra sus marcajes reales de asistencia, contando como ausencia tanto los días sin marcaje como los marcados con tipo_ausencia. El resultado es el porcentaje real de ausentismo, sin depender de una vista atada a un periodo fijo.

12.6 Rotación de personal por departamento y mes

¿Qué resuelve? Cuentas altas (contratos que iniciaron) y bajas (contratos que terminaron) por departamento y mes en dos CTEs independientes, las cruza contra la plantilla activa actual y calcula la tasa de rotación como bajas sobre empleados activos.

12.7 Ranking salarial dentro de cada puesto

¿Qué resuelve? Usa RANK(), DENSE_RANK() y PERCENT_RANK() particionados por puesto para ubicar a cada empleado frente a sus pares en el mismo cargo, útil para detectar brechas salariales dentro de un mismo puesto, no solo entre departamentos.

12.8 Top 3 empleados mejor pagados por departamento

¿Qué resuelve? Con ROW_NUMBER() particionado por departamento, identifica a los tres salarios más altos de cada área, el tipo de consulta que un gerente pide para revisar concentración salarial sin ver la nómina completa.

12.9 Antigüedad de empleados por rangos

¿Qué resuelve? Combina `fn_calcular_antigüedad` con una clasificación por CASE (menos de 1 año, 1 a 3, 3 a 5, 5 a 10, más de 10) y calcula el salario promedio de cada rango apoyándose en `fn_salario_vigente` — un ejemplo de cómo reutilizar las funciones del esquema dentro de una consulta de agregación.

12.10 Contratos próximos a vencer

¿Qué resuelve? Filtra los contratos activos cuya `fecha_fin` cae dentro de los próximos 30 días, con `DATEDIFF` para mostrar cuántos días quedan — soporte directo para que RRHH gestione renovaciones antes de que el contrato expire.

12.11 Empleados con jornada abierta

¿Qué resuelve? Detecta marcajes de entrada sin salida registrada en días ya pasados, es decir, empleados que probablemente olvidaron marcar salida. Es una consulta de control operativo, no un reporte de negocio: ayuda a limpiar inconsistencias en asistencia antes de que afecten cálculos de horas.

12.12 Empleados que ganan más que el gerente de su propio departamento

¿Qué resuelve? Un self-join a través de `departamento.id_gerente` que compara, usando `fn_salario_vigente`, el salario de cada empleado contra el de su gerente — el tipo de anomalía organizacional que vale la pena revisar aunque no rompa ninguna regla de negocio del esquema.

12.13 Auditoría de cambios: resumen por usuario y tabla

¿Qué resuelve? Agrupa auditoría por usuario y tabla afectada, cuenta cuántos INSERT/UPDATE/DELETE generó cada uno con agregación condicional (CASE dentro de SUM), y usa `RANK()` para ordenar a los usuarios por nivel de actividad — el insumo que el rol Auditor necesita para priorizar qué revisar primero, en vez de leer la tabla fila por fila.

12.14 Total de nómina por sede y departamento con subtotales

¿Qué resuelve? Usa GROUP BY ... WITH ROLLUP para mostrar el total neto por departamento, el subtotal por sede y el gran total en una sola consulta, sin tener que correr tres consultas separadas y sumarlas manualmente.

12.15 Nómina acumulada por empleado

¿Qué resuelve? Con SUM() OVER y un marco de ventana (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), muestra cuánto ha cobrado un empleado de forma acumulada a lo largo del tiempo, pago tras pago; útil para revisar el histórico de ingresos de un empleado específico.

12.16 Nota sobre rendimiento

Estas consultas se apoyan en los índices ya definidos en el esquema (idx_empleado_puesto, idx_departamento_sede, idx_puesto_departamento, idx_nomina_empleado_periodo, idx_asistencia_empleado_fecha, idx_contrato_empleado), por lo que en volúmenes grandes de datos deberían resolver sin escaneos completos de tabla. Se recomienda correr EXPLAIN sobre las consultas 12.5 y 12.6 (las que combinan más JOINS, CTEs y agregación) antes de tomarlas como evidencia final de rendimiento.

13. Implementación MongoDB

13.1 Enfoque general

A diferencia de MySQL, donde el esquema se definió con tablas fijas y relaciones estrictas, en MongoDB el diseño se organiza en colecciones de documentos, pensadas para la información de RRHH que es irregular, variable o que crece con el tiempo. La base de datos NoSQL para ITN se compone de 4 colecciones, cada una resolviendo un problema distinto que en MySQL habría requerido tablas hijas adicionales y JOINS constantes.

La relación entre MongoDB y MySQL no es una llave foránea real (los motores no se conocen entre sí), sino una llave lógica compartida: el campo id_empleado

en cada documento corresponde exactamente al empleado.id de MySQL. La consistencia entre ambos se garantiza a nivel de aplicación, no de motor.

13.2 Colección: evaluaciones

¿Qué resuelve? el módulo de evaluación de desempeño (RF06, RF07) necesita un número de criterios de evaluación que cambia según el puesto y según la política de cada año un operario de planta se evalúa por metas de producción y seguridad, mientras que un puesto administrativo se evalúa por objetivos y competencias blandas. En un modelo relacional esto obligaría a una tabla criterios_evaluacion aparte con su propia FK y un JOIN por cada consulta.

¿Por qué en documento y no en tabla? cada evaluación trae un arreglo de criterios embebido directamente en el mismo documento (nombre_criterio, puntaje, comentario), de largo variable. Se lee completa en una sola operación, sin necesidad de reconstruirla con JOINS.

Estructura conceptual:

- Identificación: empleado evaluado, evaluador, periodo semestral, fecha.
- Contenido variable: arreglo de criterios con su puntaje y comentario.
- Resultado: puntaje total calculado, estado (borrador / cerrada).
- Trazabilidad: referencia a la evaluación anterior cuando se trata de una corrección (regla de negocio 12: una evaluación cerrada no se modifica, se crea un nuevo registro referenciando al original).

Utilidad para el sistema: es la fuente del **Reporte #2 (histórico de evaluaciones por empleado)**, que se arma combinando este historial de MongoDB con los datos estructurados del empleado en MySQL (nombre, puesto, departamento). También alimenta el ranking de desempeño que después vive en Redis.

Validación de esquema: aunque MongoDB no exige columnas fijas, sí se define una validación mínima (\$jsonSchema) para obligar que todo documento tenga al menos empleado, evaluador, periodo y un arreglo de criterios no vacío esto evita que "flexible" se convierta en "sin ningún control de calidad de datos".

13.3 Colección: capacitaciones

‘Qué resuelve? el módulo de formación (RF08, RF09) necesita registrar, para cada curso, una lista de empleados inscritos con estados distintos entre sí (uno puede estar "completado", otro "en curso", otro "vencido"), y esa lista cambia de tamaño constantemente. Modelarlo en SQL implicaría una tabla intermedia `inscripcion_capacitacion` con FK a `empleado` y a `capacitacion`.

¿Por qué en documento y no en tabla?: cada capacitación es un documento que contiene su propio arreglo de inscripciones, cada una con su empleado, fecha de inscripción, estado y calificación. Consultar "quién está inscrito en este curso" es leer un solo documento, no hacer un JOIN sobre miles de registros.

Estructura conceptual:

- Datos generales del curso: nombre, tipo (obligatoria / opcional), puesto al que aplica, modalidad.
- Contenido variable: temario y duración, que cambian de un curso a otro sin una estructura común.
- Arreglo de inscripciones: empleado, fechas, estado, calificación.

Utilidad para el sistema: resuelve el **Reporte #5 (cumplimiento de capacitaciones obligatorias)**: al filtrar por tipo: "obligatoria" y el puesto correspondiente, se identifican directamente los empleados cuyo estado de inscripción no es "completado" justamente el caso de uso que RF09 pide notificar automáticamente (certificaciones próximas a vencer).

13.4 Colección: `perfiles_empleado`

¿Qué resuelve? la hoja de vida de un empleado (experiencia laboral, educación, habilidades) no tiene la misma cantidad de campos para todos: un operario recién ingresado puede no tener experiencia previa registrada, mientras que un gerente puede tener varios empleos anteriores y varios títulos. Forzar esto en columnas fijas de MySQL generaría muchas columnas vacías o tablas adicionales por cada tipo de dato variable (experiencia, educación, habilidades), cada una con su propia FK a `empleado`.

Estructura conceptual:

- `id_empleado` como llave lógica hacia MySQL.
- `experiencia_laboral`: arreglo libre de empleos anteriores.
- `educacion`: arreglo libre de estudios.

- habilidades: lista simple de competencias.

Utilidad para el sistema: sirve como repositorio de la información de reclutamiento y desarrollo profesional que no es parte del ciclo operativo diario (nómina, asistencia), pero que RRHH necesita consultar al considerar promociones o reasignaciones internas.

13.5 Colección: `bitacoras_sistema`

¿Qué resuelve? además de la tabla auditoria de MySQL (que registra cambios campo por campo sobre datos sensibles: salario anterior/nuevo, estado de contrato anterior/nuevo), el sistema necesita un registro más general de **acciones de uso** quién inició sesión, quién exportó un reporte, quién accedió a un módulo. Este tipo de evento no siempre tiene la misma forma: exportar un reporte trae un formato y un filtro aplicado, iniciar sesión trae una IP y un dispositivo, y así sucesivamente. Si cada tipo de evento tuviera que forzarse en una tabla relacional, habría que crear una tabla nueva cada vez que aparece un evento distinto.

Diferencia clave con auditoria (MySQL): auditoria es un log estructurado e inmutable de *cambios sobre un registro específico* (RNF03: no editable ni eliminable); `bitacoras_sistema` es un log de *actividad general del sistema*, con una estructura de metadatos libre según el tipo de evento.

Estructura conceptual:

- Usuario que ejecuta la acción, tipo de acción, módulo del sistema.
- Un campo metadata de forma libre, específico para cada tipo de evento.
- Fecha y hora.

Utilidad para el sistema: trazabilidad de uso general (no de datos sensibles), útil para detectar patrones de uso, accesos inusuales o para soporte técnico.

13.6 Resumen de justificación (por qué MongoDB y no otra NoSQL)

Tal como se documentó en la fase de arquitectura (sección 10.2), se descartaron Elasticsearch, DynamoDB y Cassandra porque el dominio no requiere búsqueda de texto libre a gran escala (Elasticsearch), ni un modelo de acceso por llave-valor puro sin consultas complejas (DynamoDB), ni escritura distribuida masiva multi-nodo (Cassandra). El patrón real del dominio es **documentos con**

arreglos anidados y estructura variable, que es exactamente el caso de uso central de un motor orientado a documentos como MongoDB.

14. Implementación Redis

14.1 Enfoque general

Redis no almacena información de negocio permanente: guarda **datos derivados, de acceso muy frecuente o de vida corta**, cuyo propósito es que el sistema responda rápido sin recalculer todo el tiempo contra MySQL o MongoDB. Se definieron 4 casos de uso, cada uno aprovechando una estructura de datos distinta de Redis según el problema que resuelve.

14.2 Bitmaps — control de asistencia biométrica

¿Qué resuelve? con más de 10,000 empleados marcando entrada y salida todos los días en 3 sedes, calcular ausentismo o puntualidad agregada directamente sobre la tabla asistencia de MySQL (RNF05: respuesta en menos de 5 segundos) se vuelve costoso a medida que crece el histórico.

¿Por qué un bitmap? un bitmap en Redis es, conceptualmente, una cadena de bits donde cada posición representa un día del periodo (por ejemplo, el bit 26 representa el día 26 del mes) y su valor (1 o 0) representa si el empleado marcó presente ese día. Con esto, contar cuántos días asistió un empleado en un mes es una sola operación (BITCOUNT) sobre una estructura extremadamente compacta en memoria, en lugar de un COUNT() con GROUP BY sobre millones de filas.

Utilidad adicional: al combinar los bitmaps de varios empleados de un mismo departamento con una operación bit a bit (BITOP AND), se puede calcular en milisegundos cuántos días **todo un equipo** estuvo presente simultáneamente en algo relevante para turnos de planta donde importa la cobertura conjunta, no solo individual.

Relación con el resto del sistema: este cálculo agregado alimenta al **Reporte #3 (ausentismo por periodo)** como una capa de caché/pre-cálculo rápido; el detalle fino (hora exacta de entrada/salida, tipo de ausencia) sigue viviendo en MySQL, que es quien tiene la fuente de verdad.

14.3 Sorted Sets — ranking de desempeño

¿Qué resuelve? el **Reporte #6 (ranking de desempeño)** necesita mostrar, por sede o por departamento, el top de empleados según su puntaje de evaluación es algo que se consulta con frecuencia (para bonos, promociones, reconocimientos) pero que, calculado desde cero cada vez con un ORDER BY sobre los datos de evaluación, sería una operación pesada, especialmente porque el puntaje real vive en MongoDB (estructura variable) y no en una tabla indexada de MySQL.

¿Por qué un sorted set? es una estructura de Redis pensada exactamente para esto, cada miembro (el empleado) se guarda con un score (su puntaje de evaluación), y Redis los mantiene siempre ordenados internamente. Pedir "los 5 mejores" o "la posición de un empleado específico en el ranking" es una operación directa e inmediata, sin necesidad de reordenar nada en el momento de la consulta.

¿Cuándo se actualiza? cada vez que se cierra una evaluación en MongoDB, el puntaje final se sube al sorted set correspondiente a la sede y periodo. El ranking en Redis es entonces una "vista materializada" de un subconjunto de los datos de MongoDB, optimizada solo para el caso de uso de ordenar y consultar top-N.

14.4 Caché de reportes pesados

¿Qué resuelve? reportes como la nómina consolidada por departamento agregan potencialmente miles de registros de nomina en MySQL. Si varios usuarios de Contabilidad o gerentes consultan el mismo reporte varias veces en el mismo periodo, recalcularlo cada vez desde cero es un desperdicio de recursos.

¿Cómo funciona? el resultado ya calculado del reporte se guarda en Redis como texto (por ejemplo, en formato JSON) bajo una clave que identifica el reporte, el departamento y el periodo, con un **tiempo de expiración (TTL)** definido. Mientras la clave exista, el sistema devuelve el dato cacheado en lugar de volver a consultar MySQL; cuando expira, se recalcula una sola vez y se vuelve a guardar.

Beneficio directo: ayuda a cumplir RNF05 (respuesta en menos de 5 segundos) incluso en los reportes más pesados, sin necesidad de mantener sincronizada una tabla de resumen permanente en MySQL.

14.5 Sesiones de usuario

¿Qué resuelve? cada solicitud autenticada al sistema necesita verificar quién es el usuario y qué rol tiene (RNF02: acceso restringido por roles), y esa verificación ocurre en *cada* petición, no una sola vez. Hacer esa validación contra MySQL en cada clic sería innecesariamente lento para un dato que, además, es temporal por naturaleza (una sesión expira).

¿Cómo funciona? al iniciar sesión, se guarda en Redis un registro con el token de sesión, el rol del usuario y su id_empleado, con un tiempo de vida limitado (por ejemplo, 8 horas de jornada laboral). Cada solicitud posterior valida contra Redis en vez de contra la base de datos relacional, y al expirar el tiempo, la sesión desaparece automáticamente sin necesidad de un proceso de limpieza manual.

14.6 Contadores derivados

¿Qué resuelve? valores como "empleados activos por sede" o "capacitaciones pendientes por puesto" se consultan con frecuencia (por ejemplo, en un dashboard gerencial) pero cambian constantemente. Recalcularlos con un COUNT() sobre MySQL en cada visualización del dashboard es redundante si el valor puede mantenerse actualizado de forma incremental.

¿Cómo funciona? en lugar de contar desde cero, el contador en Redis se incrementa o decrementa exactamente en el momento en que ocurre el evento que lo afecta (un alta de empleado incrementa el contador de su sede, una baja lo decrementa; completar una capacitación obligatoria decrementa el contador de pendientes de ese puesto). Leer el valor actual es entonces una operación instantánea.

14.7 Resumen de justificación (por qué Redis y no cachear en MySQL)

MySQL garantiza consistencia e integridad para los datos de negocio permanentes, pero no está optimizado para lecturas de altísima frecuencia sobre datos que cambian todo el tiempo y que no necesitan sobrevivir a un reinicio del sistema (sesiones, cachés, contadores). Redis, al operar en memoria y ofrecer estructuras especializadas (bitmaps, sorted sets) en vez de solo pares

clave-valor genéricos, resuelve estos casos puntuales de rendimiento sin comprometer la integridad transaccional que sí exige el resto del sistema.

14.8 Nota sobre consistencia entre los tres motores

Ni MongoDB ni Redis mantienen una relación referencial automática con MySQL: ambos dependen de que la aplicación mantenga sincronizado el campo `id_empleado` (o el `id` correspondiente) entre los tres motores. Esto es una decisión de diseño consciente de la arquitectura polígloa — se prioriza el rendimiento y la flexibilidad de cada motor para su tipo de dato, a cambio de mover la responsabilidad de consistencia entre motores a la capa de aplicación, en vez de forzarla a nivel de base de datos.

15. Conclusiones

El presente proyecto desarrolló el diseño e implementación de un sistema de base de datos para el departamento de Recursos Humanos de Industrial Textil Nacional (ITN), cubriendo el ciclo completo de gestión de sus colaboradores: datos personales y contractuales, nómina, evaluaciones de desempeño, formación y control de asistencia, con auditoría de cambios sobre datos sensibles e integración con los sistemas externos de contabilidad y de tiempo y asistencia definidos en los requerimientos funcionales.

Dada la naturaleza heterogénea de la información que maneja RRHH —datos altamente estructurados junto a información flexible y variable, además de necesidades de alto rendimiento en tiempo real—, el sistema se diseñó bajo una arquitectura de persistencia polígloa, combinando tres motores de base de datos según lo definido en la justificación técnica del proyecto:

- **MySQL**, para las entidades con estructura fija y relaciones estrictas (Sede, Departamento, Puesto, Empleado, Contrato, Nómina, Asistencia, Usuario y Auditoría), aprovechando las garantías ACID necesarias para que el cálculo de nómina y la integridad referencial entre módulos sean consistentes, y aplicando normalización hasta 3FN sobre el modelo relacional.
- **MongoDB**, para la información con estructura variable o anidada (evaluaciones, capacitaciones, perfiles de empleado y bitácoras del sistema), evitando forzar en columnas fijas datos que cambian de forma

y de tamaño según el empleado, el puesto o la política de evaluación de cada periodo.

- **Redis**, para los datos efímeros o de alta velocidad de consulta (bitmaps de asistencia, sorted sets de ranking de desempeño, caché de reportes pesados, sesiones de usuario y contadores derivados), resolviendo los requerimientos de rendimiento sin comprometer la integridad de los datos permanentes del negocio.

A nivel de diseño lógico y físico, se definieron las tablas, tipos de datos, llaves primarias y foráneas, constraints e índices necesarios para sostener las reglas de negocio identificadas en el levantamiento de requerimientos (por ejemplo, que un empleado no puede tener dos contratos activos simultáneamente, que un pago de nómina no puede procesarse sin contrato activo, o que todo cambio sobre datos sensibles debe quedar auditado de forma inmutable). Estas reglas se implementaron no solo como validaciones de aplicación, sino como triggers, procedimientos almacenados y funciones dentro del propio motor relacional, de modo que se cumplan sin importar desde dónde se escriba en la base de datos. Sobre este mismo esquema se construyeron vistas para los reportes complejos definidos en los requerimientos (nómina consolidada por departamento, ausentismo por periodo, rotación de personal, entre otros) y consultas avanzadas con joins múltiples, subconsultas, CTEs y funciones de ventana.

El resultado es un sistema cuyo diseño responde directamente a los requerimientos funcionales y no funcionales establecidos al inicio del proyecto: la separación de responsabilidades entre los tres motores permite atender simultáneamente la necesidad de integridad transaccional (RF01-RF05), la flexibilidad de datos variables (RF06, RF08), el rendimiento en reportes (RNF05) y la trazabilidad de cambios sensibles (RF10, RNF03), sin forzar todo el dominio dentro de un único modelo de datos.